

Linux & Ubuntu

Command Reference

The commands, flags, shortcuts and quick-reference tables you actually reach for — organized, explained, and ready to search or print.

Badge legend: ESSENTIAL = daily use INTERMEDIATE = common admin
POWER USER = advanced / composable DANGER = can remove data or alter system

File Management

ls · cp · find · grep · stat · tree ...

Users & Permissions

chmod · chown · sudo ...

Package Management

apt · dpkg · snap ...

System & Resources

top · systemctl · journalctl ...

Process Control

ps · kill · jobs · timeout ...

Shell Power Tools

history · alias · xargs · tmux ...

Networking

ssh · curl · rsync · ss ...

Disk Utilities

df · du · lsblk · mount ...

Secret Tips & Hacks

!! · Ctrl+R · fzf · watch ...

\$ File Management

ls **ESSENTIAL**

List directory contents.

syntax

```
$ ls [options] [path]
```

```
$ ls -lah
```

Long list, human sizes, incl. hidden files

```
$ ls -lt
```

Sort by newest modification time

TIP Alias ll='ls -lah' in ~/.bashrc for instant detailed listings.

cd **ESSENTIAL**

Change the current working directory.

syntax

```
$ cd [path]
```

```
$ cd -
```

Jump back to the previous directory

```
$ cd ~
```

Go to your home directory

TIP cd with no arguments always takes you home.

pwd **ESSENTIAL**

Print the full path of the working directory.

syntax

```
$ pwd
```

```
$ pwd
```

Show where you are

```
$ pwd -P
```

Resolve symlinks to the real path

TIP Use \$PWD in scripts to reference the current directory.

cp **ESSENTIAL**

Copy files and directories.

syntax

```
$ cp [options] src dst
```

```
$ cp -r src/ backup/
```

Recursively copy a directory

```
$ cp -av a b
```

Archive mode, verbose

TIP Add -i to be prompted before overwriting existing files.

mv **ESSENTIAL**

Move or rename files and directories.

syntax

```
$ mv [options] src dst
```

```
$ mv old.txt new.txt
```

Rename a file

```
$ mv *.jpg images/
```

Move all JPGs into a folder

TIP mv is atomic on one filesystem — great for safe publish swaps.

rm **ESSENTIAL** **DANGER**

Remove files and directories.

syntax

```
$ rm [options] file
```

```
$ rm -i file
```

Delete with confirmation

```
$ rm -rf build/
```

Force-remove a directory tree

TIP rm -rf is irreversible. Double-check the path before Enter. Prefer trash-cli for safety.

mkdir **ESSENTIAL**

Create directories.

syntax

```
$ mkdir [options] dir
```

```
$ mkdir -p a/b/c
```

Create nested dirs, no error if they exist

```
$ mkdir -m 700 secret
```

Create with specific permissions

TIP mkdir -p never fails on existing dirs — safe in scripts.

touch **ESSENTIAL**

Create empty files or update timestamps.

syntax

```
$ touch file
```

```
$ touch newfile
```

Create an empty file

```
$ touch -d '2 hours ago' f
```

Set a specific modification time

TIP Great for creating placeholder files or bumping mtimes.

\$ File Management (cont.)

nano **ESSENTIAL**

Simple terminal text editor (no Vim learning curve).

syntax

```
$ nano [options] file
```

```
$ nano ~/.bashrc
```

Edit your shell config

```
$ nano -w file
```

Disable line wrapping

TIP Ctrl+O save, Ctrl+X exit. Perfect for quick config edits.

file **INTERMEDIATE**

Identify what a file actually contains.

syntax

```
$ file [options] file
```

```
$ file script.sh
```

Show type: shell script, binary, etc.

```
$ file -b *
```

Brief type for every item

TIP Useful when an extension is missing or lying.

stat **INTERMEDIATE**

Display detailed file status and metadata.

syntax

```
$ stat [options] file
```

```
$ stat file.txt
```

Show size, permissions, timestamps

```
$ stat -c '%n %s %y' *
```

Custom format: name size mtime

TIP Use -c / --format for scripting; %s = size, %a = octal mode.

cat **ESSENTIAL**

Concatenate and print file contents.

syntax

```
$ cat [options] file
```

```
$ cat -n script.sh
```

Print with line numbers

```
$ cat a b > all
```

Merge files into one

TIP For large files use 'less' so you can scroll. Consider 'bat' for syntax highlighting.

less **ESSENTIAL**

View file contents one screen at a time.

syntax

```
$ less file
```

```
$ less +F app.log
```

Follow a growing file (like tail -f)

```
$ less -N file
```

Show line numbers

TIP Inside less: / searches, n/N cycle matches, q quits, g/G jump.

head **INTERMEDIATE**

Output the first part of a file.

syntax

```
$ head [options] file
```

```
$ head -n 20 file
```

First 20 lines

```
$ head -c 100 file
```

First 100 bytes

TIP Pair head with tail to slice a middle range of lines.

tail **INTERMEDIATE**

Output the last part of a file.

syntax

```
$ tail [options] file
```

```
$ tail -f app.log
```

Follow a log as it grows

```
$ tail -n 50 file
```

Last 50 lines

TIP tail -F keeps following even if the file is rotated/recreated.

find **POWER USER**

Search a tree by name, size, time and more.

syntax

```
$ find [path] [tests] [actions]
```

```
$ find . -name '*.log' -mtime +7 -delete
```

Delete logs older than 7 days

```
$ find /var -size +100M
```

Files larger than 100 MB

TIP -delete is irreversible. Preview first with -print. Prefer fd for speed + gitignore.

\$ File Management (cont.)

grep **INTERMEDIATE**

Search text using patterns (regex).

syntax

```
$ grep [options] pattern [file]
```

```
$ grep -rin error /var/log
```

Recursive, case-insensitive, line numbers

```
$ grep -v '^#' cfg
```

Show lines that are NOT comments

TIP grep -F is a faster fixed-string search; -E enables extended regex.

rg **POWER USER**

Ripgrep: fast recursive search that honors .gitignore.

syntax

```
$ rg [options] pattern [path]
```

```
$ rg TODO -t py
```

Search only Python files

```
$ rg -i timeout --stats
```

Case-insensitive with match stats

TIP Often 5-10x faster than grep -r and skips build/vendor dirs.

sed **POWER USER**

Stream editor for find/replace and transforms.

syntax

```
$ sed [options] 'script' file
```

```
$ sed -i 's/foo/bar/g' file
```

In-place replace all foo with bar

```
$ sed -n '10,20p' file
```

Print only lines 10-20

TIP Test without -i first; add -i.bak to keep a backup copy.

awk **POWER USER**

Pattern-scanning and column processing.

syntax

```
$ awk 'program' file
```

```
$ awk '{print $1,$3}' f
```

Print columns 1 and 3

```
$ awk -F: '{print $1}' /etc/passwd
```

Split on ':' and print field 1

TIP awk '{sum+=\$1} END{print sum}' totals a column of numbers.

sort / uniq **INTERMEDIATE**

Sort lines and report or filter unique ones.

syntax

```
$ sort [opts] | uniq [opts]
```

```
$ sort file | uniq -c
```

Count occurrences of each line

```
$ sort -n -k2 data.tsv
```

Numeric sort on column 2

TIP sort -u is a shortcut for sort | uniq. Use -h for human-numeric sizes.

cut **INTERMEDIATE**

Remove sections from each line of files.

syntax

```
$ cut [options] file
```

```
$ cut -d: -f1 /etc/passwd
```

First field of colon-separated file

```
$ cut -c1-10 file
```

Characters 1 through 10 of each line

TIP Often paired with sort/uniq or awk for quick column extraction.

wc **INTERMEDIATE**

Count lines, words, and bytes.

syntax

```
$ wc [options] file
```

```
$ wc -l file
```

Count lines only

```
$ wc -w *.md
```

Word count of all Markdown files

TIP wc -l is the classic 'how many lines?' one-liner.

diff **INTERMEDIATE**

Compare files or directories line by line.

syntax

```
$ diff [options] a b
```

```
$ diff -u old new
```

Unified diff (readable)

```
$ diff -rq dir1 dir2
```

Recursive, report differing files only

TIP Pipe into less or use colordiff for friendlier output.

\$ File Management (cont.)

tar **INTERMEDIATE**

Create and extract archive (tarball) files.

syntax

```
$ tar [flags] archive files
```

```
$ tar -czvf site.tgz site/
```

Create a gzip archive

```
$ tar -xzvf site.tgz
```

Extract a gzip archive

TIP Mnemonic: Create Ze Vfile → czvf; eXtract → xzvf.

ln **INTERMEDIATE**

Create hard or symbolic links.

syntax

```
$ ln [-s] target link
```

```
$ ln -s /opt/app/bin/app /usr/local/bin/app
```

Symlink a real binary onto PATH

```
$ ln -sf new current
```

Force-update a symlink

TIP Symlinks (-s) can cross filesystems; hard links cannot.

tree **INTERMEDIATE**

List contents of directories in a tree-like format.

syntax

```
$ tree [options] [path]
```

```
$ tree -L 2
```

Limit depth to 2 levels

```
$ tree -d -I 'node_modules'
```

Dirs only, ignore pattern

TIP Install with: `sudo apt install tree`. Great for quick structure overview.

\$ System & Resources

top **ESSENTIAL**

Live view of processes and resource usage.

syntax

```
$ top
```

```
$ top -o %MEM
```

Sort processes by memory

```
$ top -u www-data
```

Only one user's processes

TIP Press k to kill, M/P to sort by memory/CPU inside top.

htop **INTERMEDIATE**

Interactive, colorized process viewer.

syntax

```
$ htop
```

```
$ htop
```

Launch the interactive viewer

```
$ sudo apt install htop
```

Install on Ubuntu/Debian

TIP F5 = tree view, F6 = change sort column. No flags to memorize.

free **ESSENTIAL**

Display total, used and free memory.

syntax

```
$ free [options]
```

```
$ free -h
```

Human-readable sizes

```
$ free -s 2
```

Refresh every 2 seconds

TIP The 'available' column, not 'free', is memory truly usable.

uname **ESSENTIAL**

Print system and kernel information.

syntax

```
$ uname [options]
```

```
$ uname -a
```

All system info in one line

```
$ uname -r
```

Kernel release version

TIP Pair with 'lsb_release -a' for exact Ubuntu version + codename.

uptime **ESSENTIAL**

Show run time and load averages.

syntax

```
$ uptime
```

```
$ uptime
```

One-line summary with 1/5/15-min load

```
$ uptime -p
```

Pretty human-readable uptime

TIP Load above core count means runnable + uninterruptible tasks are waiting (CPU or I/O).

df **ESSENTIAL**

Report filesystem disk space usage.

syntax

```
$ df [options]
```

```
$ df -h
```

Human-readable free space per mount

```
$ df -i
```

Inode usage (full but bytes free?)

TIP If df says full but du disagrees, a deleted-but-open file is to blame.

systemctl **INTERMEDIATE**

Control and inspect systemd services.

syntax

```
$ systemctl [action] [service]
```

```
$ sudo systemctl restart ssh
```

Restart a service

```
$ systemctl status docker
```

State + recent logs

TIP 'enable' starts a service at boot; 'start' is this session only.

journalctl **POWER USER**

Query the systemd journal (logs).

syntax

```
$ journalctl [options]
```

```
$ journalctl -u nginx -f
```

Follow live logs for a service

```
$ journalctl --since '1 hour ago' -p err
```

Recent errors

TIP Run --disk-usage first. --vacuum-time=7d permanently deletes older journal entries.

\$ System & Resources (cont.)

dmesg

POWER USER

Print kernel ring-buffer messages.

syntax

```
$ dmesg [options]
```

```
$ dmesg -T | tail
```

Recent kernel events, human timestamps

```
$ dmesg -w
```

Watch new kernel messages live

TIP First stop for USB, disk, and hardware error diagnosis.

lsusb / lspci

INTERMEDIATE

List USB and PCI devices.

syntax

```
$ lsusb $ lspci
```

```
$ lsusb
```

Connected USB devices

```
$ lspci -nn
```

PCI devices with vendor/device IDs

TIP Essential when a new device does not appear or a driver fails to bind.

crontab

POWER USER

Schedule recurring jobs.

syntax

```
$ crontab [-e|-l]
```

```
$ crontab -e
```

Edit your scheduled jobs

```
$ crontab -l
```

List your cron jobs

TIP Format: min hour dom mon dow cmd. Use crontab.guru to build lines.

\$ Networking

ping **ESSENTIAL**

Test connectivity via ICMP echo.

syntax

```
$ ping [options] host
```

```
$ ping -c 4 google.com
```

Send exactly 4 packets

```
$ ping -i 0.2 10.0.0.1
```

Ping every 0.2s

TIP No reply != down; many hosts/firewalls drop ICMP.

curl **INTERMEDIATE**

Transfer data to/from servers over many protocols.

syntax

```
$ curl [options] url
```

```
$ curl -fsSL url -o file
```

Download quietly to a file

```
$ curl -X POST -d @body.json -H 'Content-Type: application/json' https://api.example.com
```

JSON POST

TIP Add -I for headers only; -L to follow redirects.

wget **ESSENTIAL**

Non-interactive network downloader.

syntax

```
$ wget [options] url
```

```
$ wget -c url/big.iso
```

Resume an interrupted download

```
$ wget -mkEpnP https://site
```

Mirror a site offline

TIP Use -c to continue partial downloads on flaky connections.

ssh **INTERMEDIATE**

Securely log into and run commands remotely.

syntax

```
$ ssh [user@]host [cmd]
```

```
$ ssh -i key.pem ubuntu@1.2.3.4
```

Connect with a private key

```
$ ssh -L 8080:localhost:80 host
```

Forward a local port

TIP Store hosts/keys in ~/.ssh/config so you just type 'ssh myserver'.

scp **INTERMEDIATE**

Copy files securely over SSH.

syntax

```
$ scp [options] src dst
```

```
$ scp file user@host:/tmp/
```

Upload a file

```
$ scp -r user@host:/var/www ./
```

Download a directory

TIP For large or resumable transfers, prefer rsync.

rsync **POWER USER**

Fast incremental sync; only sends changes.

syntax

```
$ rsync [options] src dst
```

```
$ rsync -avz --progress src/ host:/dst/
```

Sync to remote with progress

```
$ rsync -a --delete src/ backup/
```

Exact mirror (destructive)

TIP Always --dry-run before --delete. It permanently removes extra files in the destination.

ip **INTERMEDIATE**

Show/manage interfaces, addresses, routes.

syntax

```
$ ip [object] [command]
```

```
$ ip -c a
```

All addresses (colorized)

```
$ ip route
```

Display the routing table

TIP 'ip a' replaces ifconfig; 'ip route get 8.8.8.8' shows the path.

ss **POWER USER**

Inspect sockets — the modern netstat.

syntax

```
$ ss [options]
```

```
$ ss -tulpn
```

Listening TCP/UDP ports + owning process

```
$ ss -s
```

Socket statistics summary

TIP 'ss -tulpn' instantly answers 'what is listening on this port?'.

\$ Networking (cont.)

dig **POWER USER**

Query DNS records.

syntax

```
$ dig [options] name
```

```
$ dig +short example.com
```

Just the answer IP(s)

```
$ dig MX example.com
```

Look up mail servers

TIP `dig @1.1.1.1 name` queries a specific resolver directly.

nc **POWER USER**

Netcat: read/write raw TCP/UDP.

syntax

```
$ nc [options] host port
```

```
$ nc -zv host 22
```

Check if a port is open

```
$ nc -l 9000
```

Listen on a port for testing

TIP Great quick test for firewall/port reachability without a client.

network path **ESSENTIAL**

Quick network troubleshooting ladder.

syntax

```
$ follow the steps below
```

```
$ ip a
```

1. Do I have an address?

```
$ ip route
```

2. Do I have a gateway?

```
$ ping <gateway>
```

3. Can I reach the router?

```
$ dig example.com
```

4. Is DNS working?

```
$ nc -zv host port
```

5. Is the service reachable?

TIP Work top-down. Stop at the first step that fails — that is the layer to fix.

\$ Users & Permissions

chmod **ESSENTIAL**

Change file access permissions.

syntax

```
$ chmod [options] mode file
```

```
$ chmod +x script.sh
```

Make a script executable

```
$ chmod 644 file
```

rw for owner, read-only for others

TIP Use 755 for directories and 644 for files as a baseline.

chown **INTERMEDIATE**

Change file owner and group.

syntax

```
$ chown [options] user:group file
```

```
$ sudo chown -R www-data: /var/www
```

Hand a web root to www-data

```
$ chown :devs file
```

Change only the group

TIP Combine with find to fix files and dirs separately.

sudo **ESSENTIAL**

Run a command as another user (usually root).

syntax

```
$ sudo [options] command
```

```
$ sudo !!
```

Re-run the previous command with sudo

```
$ sudo -u postgres psql
```

Run as a specific user

TIP 'sudo -i' opens a root shell; 'sudo -k' forgets the cached password.

passwd **ESSENTIAL**

Change an account password.

syntax

```
$ passwd [user]
```

```
$ passwd
```

Change your own password

```
$ sudo passwd alice
```

Set another user's password

TIP 'sudo passwd -l user' locks password login (SSH keys may still work). -u unlocks.

useradd / adduser **POWER USER**

Create a new user account.

syntax

```
$ useradd [options] name
```

```
$ sudo useradd -m -s /bin/bash bob
```

Create with home dir + bash

```
$ sudo adduser bob
```

Interactive, friendlier wrapper

TIP On Ubuntu prefer 'adduser' — it sets up the home dir and prompts.

usermod **POWER USER**

Modify a user account.

syntax

```
$ usermod [options] user
```

```
$ sudo usermod -aG docker $USER
```

Add yourself to the docker group

```
$ sudo usermod -s /bin/zsh alice
```

Change default shell

TIP Always -aG (omit -a and you wipe other groups). docker group ≈ root access.

groups **ESSENTIAL**

Show group memberships.

syntax

```
$ groups [user]
```

```
$ groups
```

Your own groups

```
$ id -Gn bob
```

Same via id, by name

TIP New group membership only applies after a fresh login.

whoami / id **ESSENTIAL**

Show current user identity and groups.

syntax

```
$ whoami $ id
```

```
$ whoami
```

Just the username

```
$ id
```

UID, GID, and all groups

TIP id is more informative; whoami is the quick check.

\$ Users & Permissions (cont.)

umask **POWER USER**

Set default permission mask for new files.

syntax

```
$ umask [mask]
```

```
$ umask
```

Show the current mask

```
$ umask 022
```

Default 755 dirs / 644 files

TIP umask masks off permission bits — 022 removes write for group/others.

\$ Process Control

ps **ESSENTIAL**

Report a snapshot of current processes.

syntax

```
$ ps [options]
```

```
$ ps aux
```

Every process with user/CPU/mem

```
$ ps -ef --forest
```

Process hierarchy as a tree

TIP Pipe into grep to find one: `ps aux | grep nginx`.

kill **INTERMEDIATE**

Send a signal to a process.

syntax

```
$ kill [-signal] pid
```

```
$ kill 4521
```

Ask a process to stop (SIGTERM)

```
$ kill -9 4521
```

Force-kill (SIGKILL)

TIP Try TERM first; reserve -9 for processes that won't quit.

pkill **INTERMEDIATE**

Signal processes by name.

syntax

```
$ pkill [options] pattern
```

```
$ pkill -f 'node server.js'
```

Kill by matching the full command

```
$ killall firefox
```

Kill every firefox process

TIP Preview first with `'pgrep -a pattern'` to see what you'd hit.

jobs **INTERMEDIATE**

Manage background/foreground jobs.

syntax

```
$ jobs    $ bg %n    $ fg %n
```

```
$ bg %1
```

Resume job 1 in the background

```
$ fg %1
```

Bring job 1 to the foreground

TIP Ctrl+Z suspends a program; then 'bg' keeps it running detached.

nohup **POWER USER**

Run a command that survives logout.

syntax

```
$ nohup command &
```

```
$ nohup ./backup.sh > out.log 2>&1 &
```

Detach a long job + log

```
$ disown -h %1
```

Detach an already-running job

TIP For anything long-lived, prefer tmux or a systemd unit.

lsof **POWER USER**

List open files and the processes using them.

syntax

```
$ lsof [options]
```

```
$ lsof -i :8080
```

What is using port 8080

```
$ lsof +D /mnt
```

Who holds files under a mount

TIP Essential when 'device is busy' blocks an unmount.

nice / renice **POWER USER**

Run or adjust process CPU priority.

syntax

```
$ nice/renice [priority] cmd
```

```
$ nice -n 10 ./job.sh
```

Start a job at lower priority

```
$ renice -n 5 -p 4521
```

Set niceness of PID 4521 to 5

TIP Higher niceness = lower priority. Range is -20 (highest) to 19 (lowest).

timeout **INTERMEDIATE**

Run a command with a time limit.

syntax

```
$ timeout [duration] command
```

```
$ timeout 30s ./longjob
```

Kill after 30 seconds

```
$ timeout -k 5 10s cmd
```

SIGTERM then SIGKILL after 5s more

TIP Useful in scripts and CI to prevent hung processes.

\$ Disk Utilities

du **INTERMEDIATE**

Estimate file/directory space usage.

syntax

```
$ du [options] [path]
```

```
$ du -sh *
```

Size of each item here

```
$ du -h --max-depth=1 / | sort -rh | head
```

Top space users

TIP 'du -sh * | sort -h' quickly reveals what's eating disk.

lsblk **INTERMEDIATE**

List block devices in a tree.

syntax

```
$ lsblk [options]
```

```
$ lsblk -f
```

Show filesystems, labels, UUIDs

```
$ lsblk -o NAME,SIZE,MOUNTPOINT
```

Custom columns

TIP Run before/after plugging a USB to spot its device name.

mount **POWER USER**

Attach a filesystem to a directory.

syntax

```
$ mount [device] [dir]
```

```
$ sudo mount /dev/sdb1 /mnt
```

Mount a partition

```
$ mount | column -t
```

List current mounts aligned

TIP Add to /etc/fstab for auto-mount; test with 'mount -a' first.

umount **POWER USER**

Detach a mounted filesystem.

syntax

```
$ umount [dir|device]
```

```
$ sudo umount /mnt
```

Unmount a mount point

```
$ sudo umount -l /mnt
```

Lazy unmount if busy

TIP If 'target is busy', find the holder with 'lsdf +D /mnt'.

ncdu **INTERMEDIATE**

Interactive disk usage explorer.

syntax

```
$ ncdu [path]
```

```
$ ncdu /
```

Explore usage folder by folder

```
$ sudo apt install ncdu
```

Install on Ubuntu/Debian

TIP Navigate with arrows. 'd' deletes the item — there is no undo. Use carefully.

fdisk **POWER USER** **DANGER**

Inspect and edit disk partitions.

syntax

```
$ sudo fdisk [-l] [device]
```

```
$ sudo fdisk -l
```

List all disks and partitions

```
$ sudo fdisk /dev/sdb
```

Interactively partition a disk

TIP Double-check the device name — partitioning the wrong disk wipes it.

mkfs **POWER USER** **DANGER**

Create a filesystem on a partition.

syntax

```
$ sudo mkfs.type device
```

```
$ sudo mkfs.ext4 /dev/sdb1
```

Format as ext4

```
$ sudo mkfs.vfat -F32 /dev/sdb1
```

Format as FAT32

TIP Formatting erases the partition — verify the target first.

\$ Package Management

apt **ESSENTIAL**

High-level Ubuntu/Debian package manager.

syntax

```
$ apt [command] [pkg]
```

```
$ sudo apt update && sudo apt upgrade
```

Refresh + upgrade everything

```
$ sudo apt install nginx
```

Install a package

TIP 'apt' is for humans; use 'apt-get' in scripts for stable output.

apt search / show **ESSENTIAL**

Find and inspect packages.

syntax

```
$ apt search|show pattern
```

```
$ apt search image editor
```

Search names + descriptions

```
$ apt show gimp
```

Version, size, dependencies

TIP 'apt-cache policy pkg' shows the exact version you'd get.

dpkg **INTERMEDIATE**

Low-level Debian package tool.

syntax

```
$ dpkg [options] pkg.deb
```

```
$ sudo dpkg -i app.deb
```

Install a local .deb

```
$ dpkg -L nginx
```

List files a package installed

TIP After a failed 'dpkg -i', run 'sudo apt -f install' to fix deps.

snap **INTERMEDIATE**

Manage sandboxed Snap packages.

syntax

```
$ snap [command] [pkg]
```

```
$ sudo snap install code --classic
```

Install with classic confinement

```
$ snap list
```

List installed snaps

TIP 'snap refresh' updates all; snaps auto-update by default.

apt clean / autoremove **INTERMEDIATE**

Reclaim space from packages.

syntax

```
$ apt autoremove $ apt clean
```

```
$ sudo apt autoremove
```

Remove unused dependencies

```
$ sudo apt clean
```

Clear the downloaded .deb cache

TIP Run these after big upgrades to free gigabytes of cache.

add-apt-repository **POWER USER**

Add third-party software repositories.

syntax

```
$ add-apt-repository ppa:...
```

```
$ sudo add-apt-repository ppa:name/ppa
```

Add a PPA

```
$ sudo add-apt-repository --remove ppa:name/ppa
```

Remove it

TIP Run 'sudo apt update' after adding a repo to pull its lists.

\$ Shell Power Tools

man / type **ESSENTIAL**

Read the manual or locate a command.

syntax

```
$ man cmd    $ type cmd
```

```
$ man ls
```

Full manual page for ls

```
$ type python
```

Show if it's alias, builtin, or path

TIP type -a shows every match. man -k keyword searches the whatis database.

history **ESSENTIAL**

Show and reuse command history.

syntax

```
$ history [n]
```

```
$ history | grep ssh
```

Find past ssh commands

```
$ !42
```

Re-run command number 42

TIP '!' reuses the last argument: mkdir foo; cd !\$

alias **INTERMEDIATE**

Create shortcuts for long commands.

syntax

```
$ alias name='command'
```

```
$ alias gs='git status'
```

Quick git status

```
$ alias ..='cd ..'
```

Go up a directory faster

TIP Put aliases in ~/.bashrc so they persist across sessions.

xargs **POWER USER**

Build command lines from piped input.

syntax

```
$ cmd | xargs [options] cmd2
```

```
$ find . -name '*.tmp' -print0 | xargs -0 -r rm --
```

Safe delete (handles spaces)

```
$ cat urls | xargs -P4 -n1 curl -O
```

Download 4 in parallel

TIP Always prefer -print0 | xargs -0 for filenames with spaces or odd chars.

tmux **POWER USER**

Terminal multiplexer: persistent, splittable sessions.

syntax

```
$ tmux [command]
```

```
$ tmux new -s work
```

Start a named session

```
$ tmux attach -t work
```

Reattach after disconnect

TIP tmux keeps sessions alive after logout — ideal for remote jobs.

env / export **INTERMEDIATE**

Show or set environment variables.

syntax

```
$ env    $ export NAME=val
```

```
$ export PATH=$PATH:/opt/bin
```

Add a dir to PATH for this session

```
$ env | grep PROXY
```

Inspect a variable

TIP Persist exports in ~/.bashrc or ~/.profile.

chained ops **INTERMEDIATE**

Chain commands by success/failure.

syntax

```
$ cmd1 && cmd2 || cmd3
```

```
$ make && ./run || echo FAILED
```

Run only if build succeeds

```
$ cmd1; cmd2
```

Run both regardless

TIP && = on success, || = on failure, ; = always next.

tee **INTERMEDIATE**

Write output to a file AND the screen.

syntax

```
$ command | tee file
```

```
$ make 2>&1 | tee build.log
```

See output and save it

```
$ echo x | sudo tee /etc/f
```

Write to a root-owned file

TIP tee -a appends instead of overwriting.

\$ Secret Tips & Hacks

bang-bang **INTERMEDIATE**

History expansion that saves huge typing.

syntax

```
$ !! !$ !*
```

```
$ sudo !!
```

Repeat last command with sudo

```
$ ls -la !$
```

Reuse the last argument

TIP '!' expands to ALL arguments of the previous command.

Ctrl+R **ESSENTIAL**

Incremental reverse-search of history.

syntax

```
$ Ctrl + R
```

```
$ Ctrl+R then 'tar'
```

Find your last tar command

```
$ Ctrl+R again
```

Cycle to older matches

TIP Press → or End to edit a match before running it.

cd - **ESSENTIAL**

Toggle between the last two directories.

syntax

```
$ cd -
```

```
$ cd -
```

Jump back where you came from

```
$ pushd / popd
```

Keep a stack of directories

TIP pushd/popd let you bookmark and pop directories in order.

fzf **POWER USER**

General-purpose fuzzy finder.

syntax

```
$ fzf
```

```
$ vim $(fzf)
```

Fuzzy-pick a file to edit

```
$ cd $(find . -type d | fzf)
```

Fuzzy-jump to a subdir

TIP Install its shell bindings to turn Ctrl+R into fuzzy history.

watch **INTERMEDIATE**

Re-run a command and watch it change.

syntax

```
$ watch [options] command
```

```
$ watch -n1 'df -h /'
```

Refresh disk usage each second

```
$ watch -d 'ls -l'
```

Highlight what changed

TIP -d highlights differences — great for a growing counter/log.

tldr **ESSENTIAL**

Example-driven simplified man pages.

syntax

```
$ tldr command
```

```
$ tldr tar
```

The common tar examples instantly

```
$ tldr --update
```

Refresh the local cache

TIP When 'man' overwhelms, 'tldr' gives the 5 commands you need.

reset / stty **ESSENTIAL**

Fix a broken/garbled terminal.

syntax

```
$ reset $ stty sane
```

```
$ reset
```

Restore a scrambled terminal

```
$ stty sane
```

Repair input after a binary dump

TIP Type it blind even if the screen looks broken, then Enter.

\$ Quick Reference Tables

How to read this reference

\$	Shell prompt — do not type the \$
[opt]	Optional value or flag
	Pipe: send output of left into right
--	End of options (protects filenames starting with -)
echo \$?	Exit status of previous command (0 = success)
&&	Run next only on success / only on failure

Permission modes (chmod)

Octal	Symbolic	Permission
7	rwX	read + write + execute
6	rw-	read + write
5	r-X	read + execute
4	r--	read only
0	---	no access
755	rxr-xr-x	standard for directories / scripts
644	rw-r--r--	standard for regular files
600	rw-----	private files (keys, secrets)

Example: chmod 754 file

7 = owner rwx · 5 = group r-x · 4 = others r--

Common signals (kill)

Signal	Number	Meaning
SIGHUP	1	hang-up; often 'reload config'
SIGINT	2	interrupt (Ctrl+C)
SIGKILL	9	force kill; cannot be caught
SIGTERM	15	polite terminate (default)
SIGSTOP	19	pause a process
SIGCONT	18	resume a paused process

Glob / wildcard patterns

*	zero or more chars (excludes leading dot by default)
?	exactly one character
[abc]	one of a, b, or c
[0-9]	one digit in the range
{a,b}	brace expansion: a and b
**	recursive (globstar; shopt -s globstar)

Redirection & pipes

> file	overwrite stdout to file
>> file	append stdout to file
2> file	redirect stderr
2>&1	merge stderr into stdout
&> file	both stdout + stderr to file
< file	feed file as stdin
	pipe stdout into next command

Terminal Keyboard Shortcuts

Ctrl + C	Send SIGINT to the foreground process
Ctrl + Z	Suspend process (resume with fg / bg)
Ctrl + D	Send EOF on empty line (often logs out)
Ctrl + R	Reverse-search command history
Ctrl + L	Clear the screen (like 'clear')
Ctrl + A	Move cursor to start of line
Ctrl + E	Move cursor to end of line
Ctrl + U	Cut from cursor to line start
Ctrl + K	Cut from cursor to line end
Ctrl + W	Delete the word before the cursor
Ctrl + Y	Paste (yank) the last cut text
Tab	Auto-complete commands, files, paths

\$ Alphabetical Index

Command → page. Use Ctrl+F or this list. Section bookmarks are also available.

add-apt-repository	14	head	3	rsync	8
alias	15	history	15	scp	8
apt	14	htop	6	sed	4
apt clean / autoremo...	14	ip	8	snap	14
apt search / show	14	jobs	12	sort / uniq	4
awk	4	journalctl	6	ss	8
bang-bang	16	kill	12	ssh	8
cat	3	less	3	stat	3
cd	2	ln	5	sudo	10
cd -	16	ls	2	systemctl	6
chained ops	15	lsblk	13	tail	3
chmod	10	lsof	12	tar	5
chown	10	lsusb / lspci	7	tee	15
cp	2	man / type	15	timeout	12
crontab	7	mkdir	2	tlldr	16
Ctrl+R	16	mkfs	13	tmux	15
curl	8	mount	13	top	6
cut	4	mv	2	touch	2
df	6	nano	3	tree	5
diff	4	nc	9	umask	11
dig	9	ncdu	13	umount	13
dmesg	7	network path	9	uname	6
dpkg	14	nice / renice	12	uptime	6
du	13	nohup	12	useradd / adduser	10
env / export	15	passwd	10	usermod	10
fdisk	13	ping	8	watch	16
file	3	pkill	12	wc	4
find	3	ps	12	wget	8
free	6	pwd	2	whoami / id	10
fzf	16	reset / stty	16	xargs	15
grep	4	rg	4		
groups	10	rm	2		